TEST-DRIVE & CAR-BUYING PLATFORM

VirtualDrive

Project analysis: architecture, features, data model and next steps



car-demo · Next.js 16 · React 19 · MongoDB

A full-stack car showroom, built in about a month

~9.3K

lines of TypeScript / TSX

22

pages (customer + admin)

20

API route handlers

3

MongoDB databases: users, catalogue, admin

Customer side

Browse brands and models, filter by body type, fuel and budget, spin a 360° view, recolour the car and book a test drive or pay a booking amount.

Admin side

A separate dashboard to manage brands, models, variants and sub-variants, with its own login.

Where it stands

Core flows work end to end. Open items are security hardening, tests and removing hard-coded demo content.

Tech stack

Next.js 16.3.5

App Router, server components, route handlers

React 19 + TypeScript 5

29 client components, typed catalogue models

Tailwind CSS 4

Utility styling via the PostCSS plugin

MongoDB driver 6.17

Direct driver, no ORM; indexes created in code

bcryptjs + Node crypto

Password hashing and HMAC-signed sessions

Razorpay (REST)

Orders API called with fetch, HMAC payment check

Also: lucide-react icons, ESLint 9 with eslint-config-next. Images come from /public and images.unsplash.com.

Architecture at a glance

Browser

- Server-rendered pages
- 29 client components: navbar, forms, 360° viewer, checkout
- Razorpay checkout widget
- httpOnly session cookies

Next.js app (app/)

- 22 pages in the App Router, catalogue pages query MongoDB directly (force-dynamic)
- 20 route handlers: /api/auth, /api/admin, /api/catalog, /api/bookings, /api/razorpay
- app/lib: auth, bookings, Mongo clients, 360° and recolour helpers

External services

- MongoDB: users db, catalogue db, admin db
- Razorpay: order creation and signature check
- Unsplash: remote images

The customer journey

1. Home

Hero, trust strip, FAQ, inventory carousel

2. Browse

/cars with brand, type, fuel, budget and search filters

3. Explore

Gallery, 360° viewer and live colour preview

4. Configure

Variant and sub-variant, colour choice

5. Commit

Test drive form, or ₹1,000 booking via Razorpay

Guard rails: login is required to book; a user cannot book the same variant twice with the same action, and the Razorpay order is refused before payment if it would be a duplicate.

Data model

Brand

name, slug, logo, sales and service-centre counts

Model

type, fuels, price range, specs, dimensions, images, brochure, published flag

Variant

fuel, transmission, price, specs

Sub-variant

engine, paint, colours, price override (embedded in Variant)

Users (own database)

- Unique email index, bcrypt password hash
- Embedded bookings[]: action, reference, car, contact, date, time, location, paymentId

Admins (own database)

Separate collection and cookie; passwords hashed with bcrypt.

Catalogue collections are linked by string ids (brandId, modelId), with unique slug indexes on brands and models.

Standout features

360° viewer

38 Brezza frames are discovered on the server from public/carImages and scrubbed on a canvas.

In-browser recolour

Body-coloured pixels are repainted per chosen colour, with a second tone for the roof on dual-tone cars.

Atomic duplicate guard

One updateOne with $not/$elemMatch pushes a booking, so two simultaneous requests cannot both save.

Notifications and mega menu

A notification dropdown and a brand-aware inventory mega menu in a 500-line navbar.

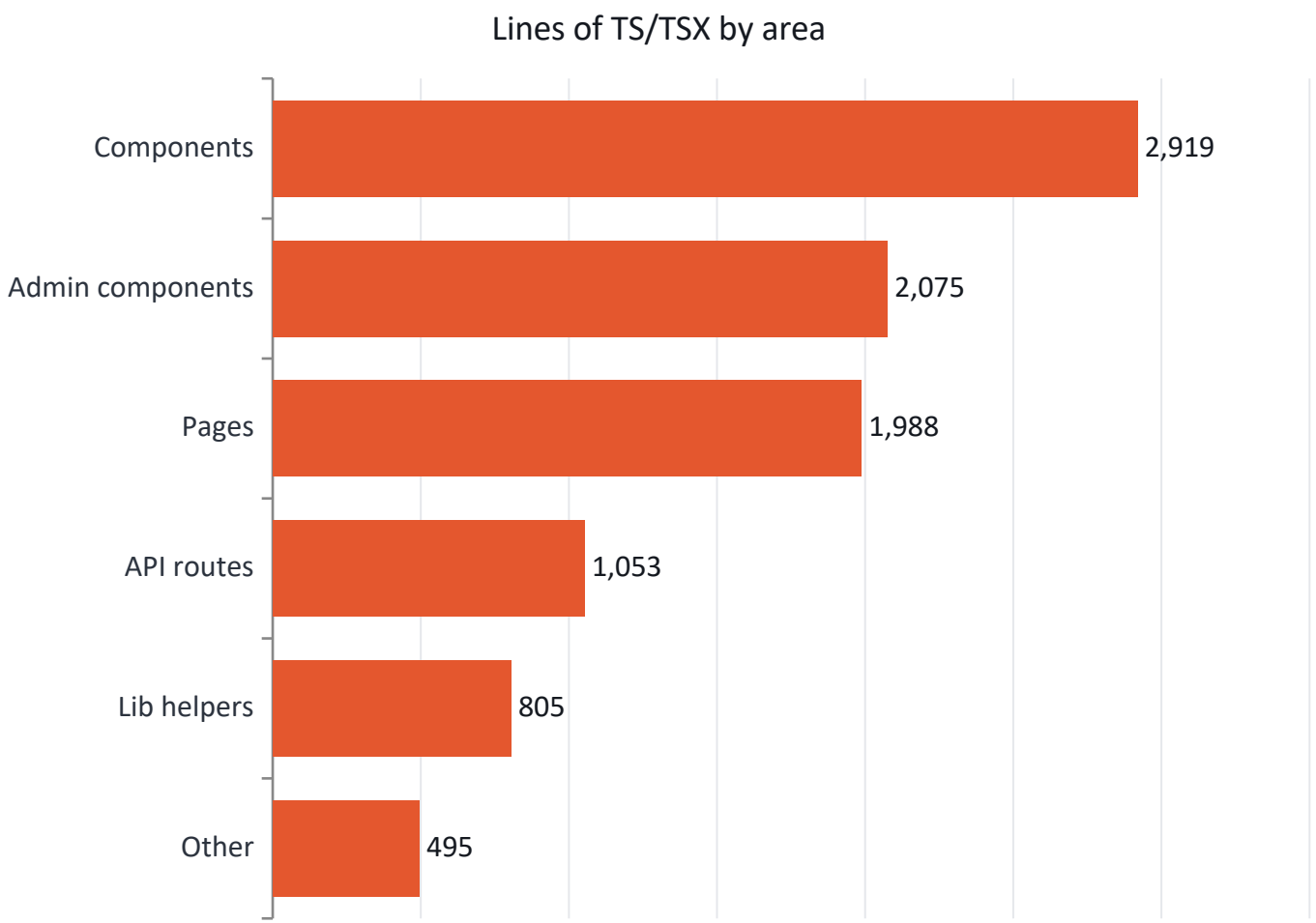
Admin dashboard

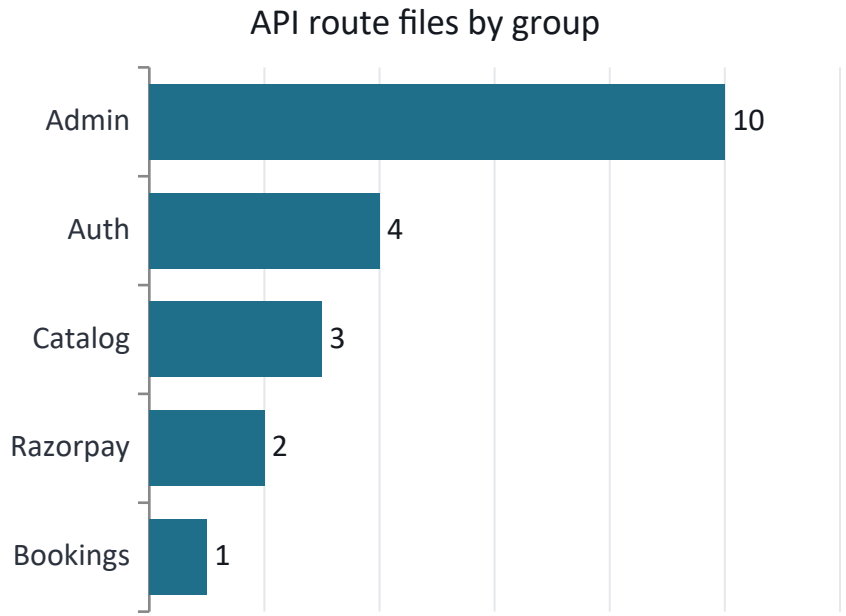
Brands	Create, edit and delete; logo, established year, sales and service-centre figures
Models	398-line form: segment, drivetrain, dimensions, capacity, images, brochure, publish toggle
Variants	Per-model variants with fuel, transmission, specs and price
Sub-variants	Engine, paint, colours and price overrides edited inline

Under the hood

- Route group app/admin/(dashboard) guarded in its layout
- 10 admin API routes, each calling getAdminUser()
- Separate admin_session cookie and admins collection
- Breadcrumbs and drill-down: brand → model → variant

Where the code lives





Admin is half the API: every catalogue write goes through it. Customer-facing reads are only 3 catalogue routes, because most pages query MongoDB directly on the server.

What is done well



Sessions

HS256-signed tokens with a 7-day expiry, timing-safe signature check, httpOnly + sameSite cookies, secure in production.



Passwords

bcrypt hashing, with a 72-byte guard so long passwords cannot be silently truncated.



Payments

Server creates the order and verifies the HMAC signature; secrets never reach the client.



Input handling

Request bodies are trimmed and length-capped through cleanString; JSON parse errors return 400.

Findings to fix first

High	**Anyone can become an admin**	POST /api/admin/signup has no invite code or existing-admin check.
High	**User and admin tokens are interchangeable**	Both are signed with the same secret and carry no role. A user can copy their session token into the admin_session cookie.
Medium	**Payment verify trusts the client**	carId, carName and price come from the request body; the order is not matched to the car or amount.
Low	**Index calls on every request**	getXCollection() runs createIndex each time, and two separate MongoClient setups exist.

Other gaps

No automated tests

package.json has no test script and no test files are tracked.

Boilerplate README

Still the create-next-app text: no setup steps or the required env vars (MONGODB_URI, MONGODB_DB, MONGODB_CATALOG_DB, MONGODB_ADMIN_DB, AUTH_SECRET, RAZORPAY_*).

Hard-coded demo content

A dedicated /maruti-suzuki/breeza page and Brezza-only 360° frames sit outside the catalogue.

No middleware or rate limiting

Login, signup and booking endpoints can be called without throttling.

Recommended next steps

1. **Lock down admin access** — Close open signup, add a role claim to tokens, check it in every admin entry point.
2. **Harden payments** — Store the order against the user and car at creation; verify against it instead of trusting the body.
3. **Add tests and a real README** — Cover auth, bookings and the duplicate guard; document env vars and seeding.
4. **Consolidate data access** — One shared Mongo client, indexes created once at startup.
5. **Make the demo data-driven** — Move the Brezza page and 360° frames behind the catalogue.

The foundation is solid; harden it before it faces users

Strong

Clean separation of catalogue, users and admins; polished 360° and recolour features.

Fix first

Open admin signup and interchangeable tokens.

Then

Tests, README, shared data layer.